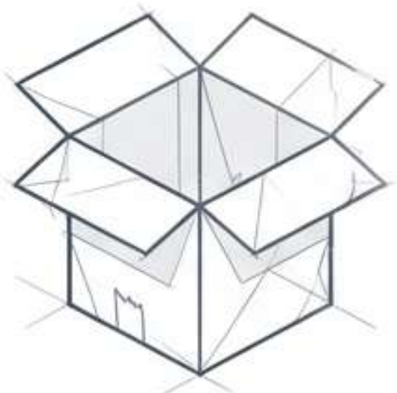
Next.js 16: Moving from Mock Data to a Real Database

PostgreSQL, Prisma, and the Persistence Layer

Codezela Career Accelerator | Full-Stack Developer Pathway |
Next.js Class — Session 03 |
Instructor: Sayuru Amarasinghe

NotebookLM

Where We Were (Sessions 1 & 2)



State: `Pretending`.

Data Source: Hardcoded local arrays & mock files.

Action Handling: Temporary UI states; route handlers that reset on refresh.

Verdict: The app looks real, but but the data disappears.

Where We Are Going (Session 3)

State: `Persisting`.

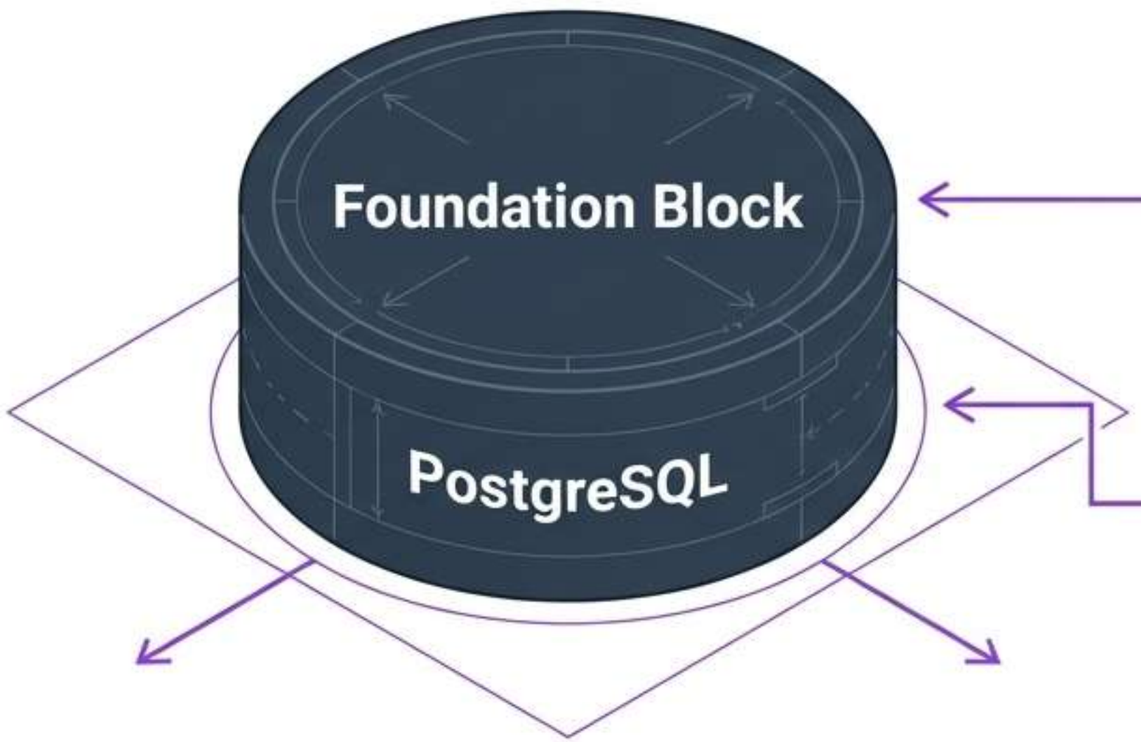
Data Source: PostgreSQL tables queried through Prisma ORM.

Action Handling: True database writes; data survives restarts.

Verdict: A production-ready, data-driven system.

NotebookLM

The Storage Engine: Why PostgreSQL?



The Engine

Reliable, widely-used relational database.
Strong for structured data.

The Content

Stores `Jobs`, `Applications`, `Users`, `Recruiters`, and `Admin actions`.

The Setup (Today)

Running locally.

Note: While Docker or Cloud setups exist for production, local execution removes friction for learning.

Without a real database, most business applications remain incomplete no matter how good the UI looks.

NotebookLM

The Translator: Understanding Prisma ORM

The Database (SQL)

```
SELECT *
FROM applications
JOIN jobs ON
  applications.job_id
  = jobs.id;
```

Prisma (The ORM Bridge)

Prisma (The ORM Bridge)

Object Relational Mapping

The Application (TypeScript / JavaScript)

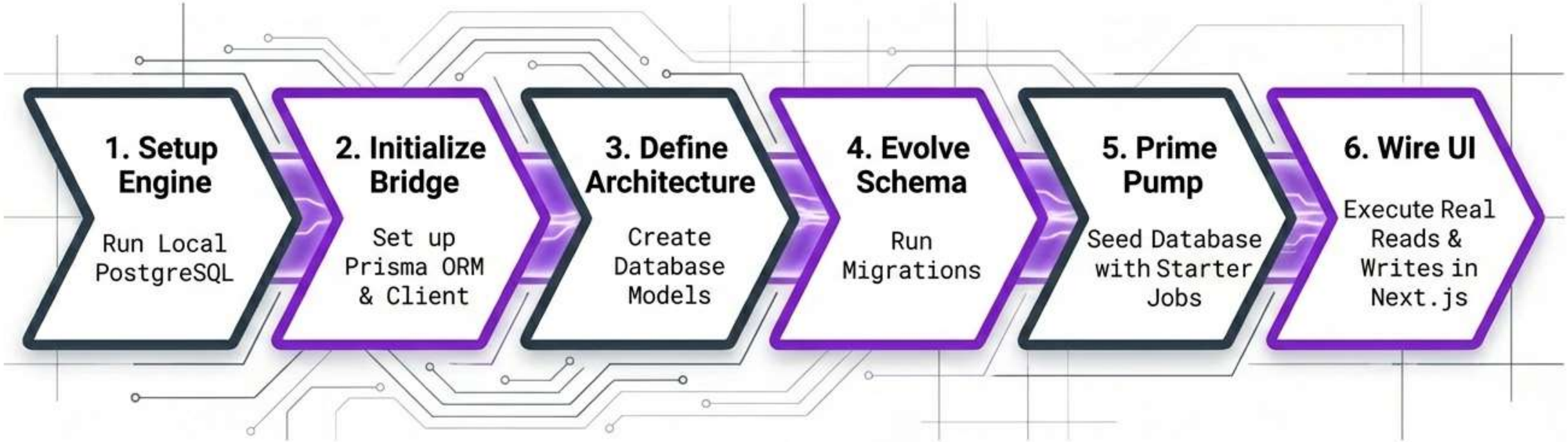
```
prisma.job.findMany()
```

Prisma replaces raw SQL complexity with code-friendly models. It allows us to interact with relational tables safely and cleanly directly from our Next.js environment.

NotebookLM

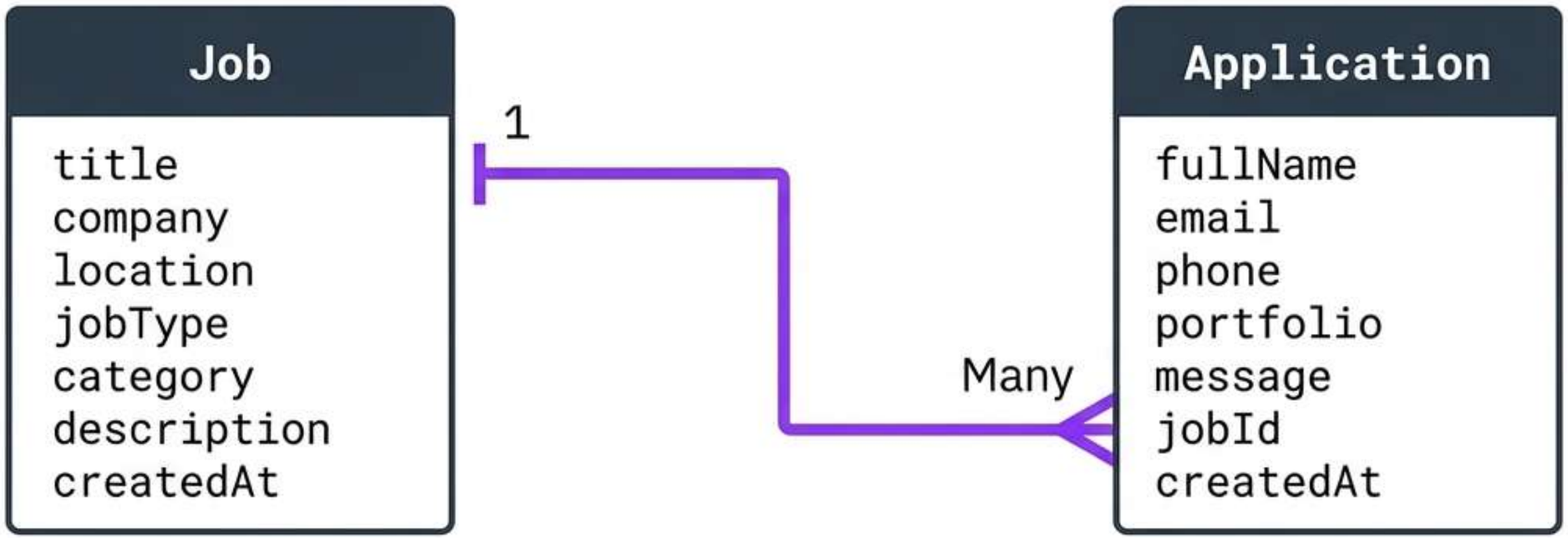
Today’s Build Pipeline

The Database Stack: Next.js 16 + PostgreSQL + Prisma Migrate + Prisma Studio.



NotebookLM

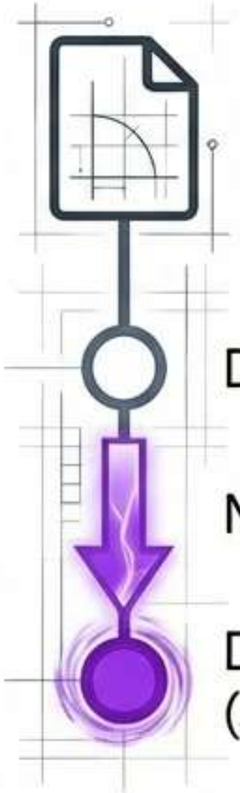
Designing the Architecture: Relational Models



One Job → Many Applications. This relationship turns disconnected data into a proper system.

NotebookLM

Schema Evolution & Migrations



`prisma/schema.prisma`
The Master Blueprint

Database v0 (Empty)

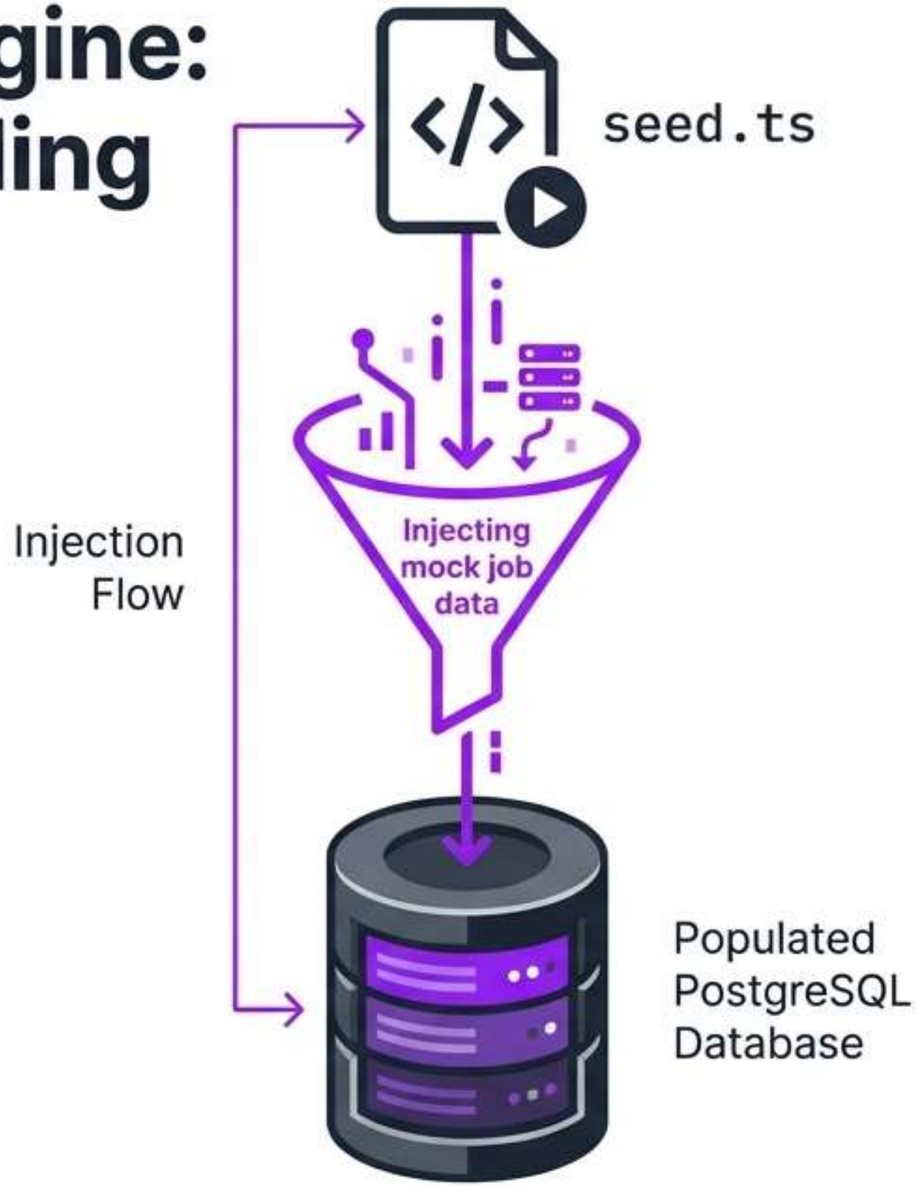
Migration Script Executed

Database v1
(Job & Application Tables Created)

What is a migration? It is a controlled, safe, and trackable database change. Think of it as version control for your database structure.

NotebookLM

Priming the Engine: Database Seeding

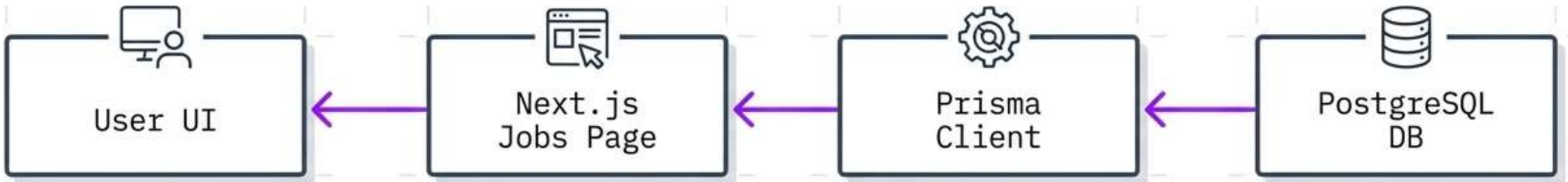


Why seed?
Instead of manually typing data into an empty app, seeding inserts realistic starter instantly.
This ensures the ensures the Jobs page has real database records to render immediately, speeding up development and testing.

NotebookLM

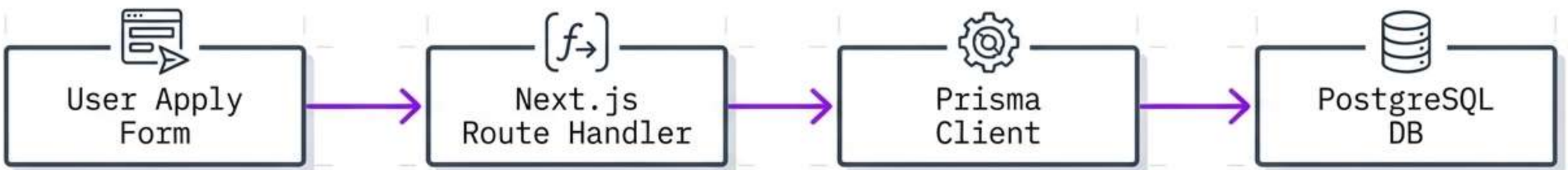
The Complete Persistence Lifecycle

The Read Lifecycle



The page fetches real jobs by ID. Search, filter, and sort now operate on actual DB records.

The Write Lifecycle

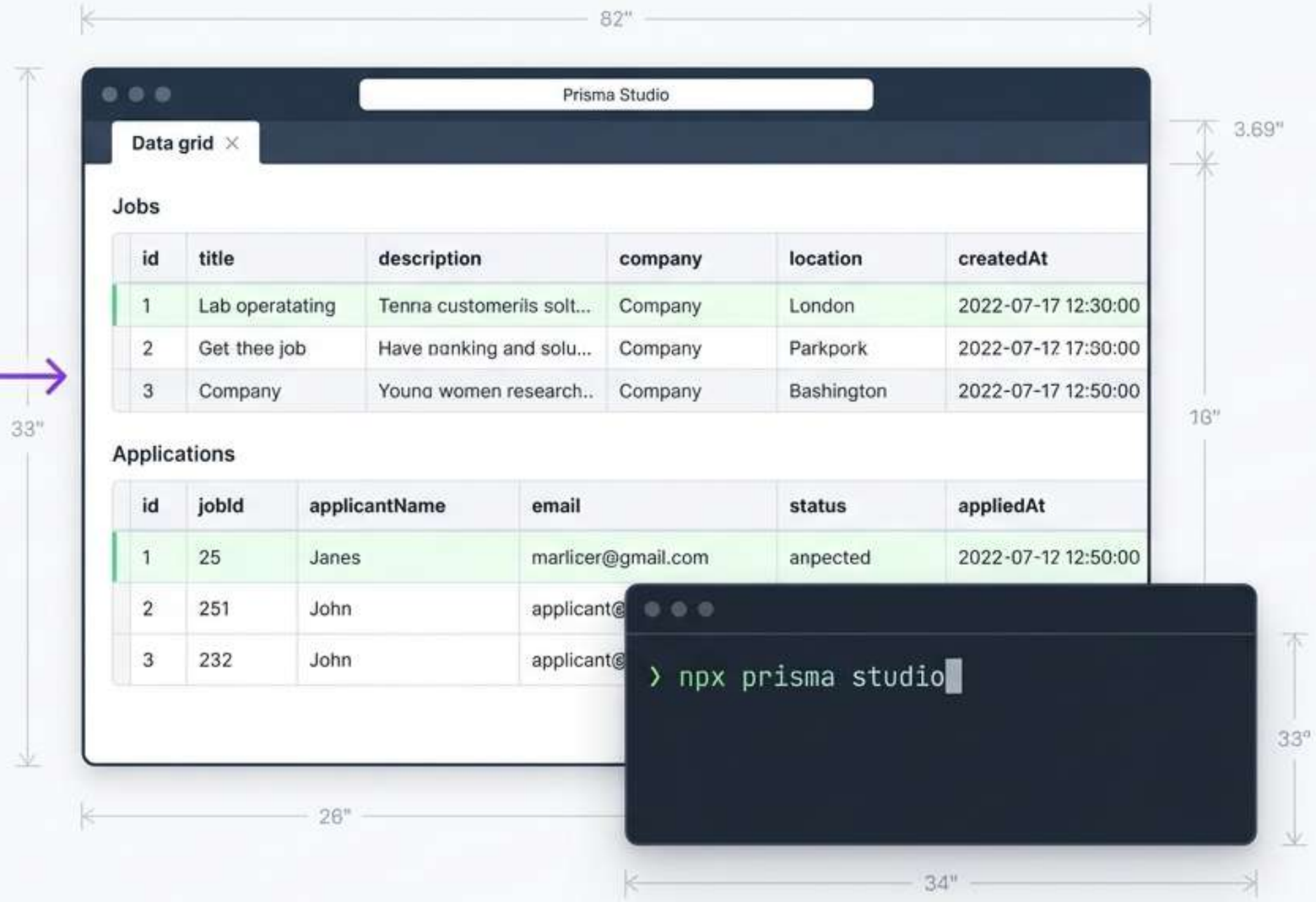


The form submits. The handler receives. Prisma creates a permanent application record.

NotebookLM

Visualizing the Unseen: Prisma Studio

Stop trusting terminal output. Prisma Studio provides a visual interface to inspect records, verify relationships, and prove that the data sent from the Next.js form is permanently stored in the PostgreSQL database.



NotebookLM